

Script Sentinel — Build Roadmap

10 slices. Slice 0 is a 7-day distribution validation gate (not interview-based — adapted to fast-ship workflow). Slices 1–9 are bounded implementation slices, each independently testable and Codex-evaluable.

Timeline pressure: 61 days until June 30, 2026 cutoff. April 15 editing block is in 17 days. The audit MVP (Slices 1–4) must be live and selling within 14 days to capture the panic window.

Slice 0 — Distribution Validation Gate (7 days, mostly no code)

Goal. Confirm you can reach Shopify Plus merchants at scale before building the audit engine. Different validation than Stocky Exodus — the buyer here is wealthy and willing to pay; the question is whether you can find them in time.

Non-goals. No customer interviews. No app scaffolding. No Script parsing logic.

What gets built.

- One-page landing site at the working domain (Framer or Carrd, ~2 hours)
- Email capture for “audit waitlist — first 50 stores get \$99 off”
- One technical post in the Shopify Plus Community Scripts Deprecation AMA category — not a sales pitch, a substantive “here’s what regression testing actually means for Script migration” piece
- Cheap Google Ads on `shopify scripts deprecation`, `shopify functions migration`, `shopify scripts june 2026`, `script to function migration`
- One LinkedIn post in Shopify Plus partner groups
- One submission to Shopify Plus Partners Slack (if accessible) or DM-outreach to 5 publicly listed Plus partner agencies
- A 90-second Loom showing a mockup of the Migration Risk PDF (what merchants will receive)

Acceptance criteria — minimum bar to proceed.

1. **30 waitlist signups in 7 days** from organic + paid combined. CPA target under \$15 from Google Ads.
2. **At least 3 of the 30 are verified Shopify Plus stores** (verifiable via myshopify.com domain, public store, or LinkedIn confirmation of role).
3. **At least one inbound DM or email from a Plus merchant** asking when the product launches or whether it's available now.
4. **Google Ads CTR above 2%** on the migration keywords. Sub-2% means the messaging isn't landing.

Browser proof. Screenshots of: Google Ads dashboard with CTR + spend, waitlist signup list with verified Plus stores marked, Shopify Community post with view count, any inbound messages.

Codex evaluation questions.

- Are signups verifiable as Plus merchants or are they tire-kickers?
- Is the Google Ads CPA sustainable at \$199 audit price (need <\$50 CPA for unit economics)?
- Did the Shopify Community post receive any organic engagement, or did it sink?
- Are there inbound questions you can't answer that reveal product-shape gaps?

Stop/go. Pass = proceed to Slice 1. Fail = either the messaging is wrong (rewrite landing page, retry) or the audience can't be reached at acceptable CPA (kill the project, distribution would bleed you out before June 30).

Time budget. 7 days. Start to stop/go decision.

Slice 1 — Plus-Only App Shell + Managed Billing

Goal. Embedded Shopify app installable on a Plus development store, with Shopify Managed Billing wired up for both one-time charges (audit) and recurring (regression suite).

Non-goals. No business logic. No Script discovery. No real UI beyond a placeholder dashboard.

User-visible behavior. Merchant clicks Install. OAuth completes. Embedded app loads inside Shopify Admin. Dashboard shows: shop name, plan tier verified as Plus (gates the entire app), and a "Run your migration audit" CTA leading to a paywall placeholder.

Plus-gate enforcement. On install, query the shop's plan via GraphQL

`Shop.plan.displayName` . If not Plus, show a friendly message ("Script Sentinel is for Shopify Plus merchants — Scripts only run on Plus") and block further use. Do not waste install slots on non-Plus stores.

Files likely to change.

- `app/shopify.server.ts` — auth + billing config (one-time + recurring)
- `app/routes/app._index.tsx` — dashboard placeholder
- `app/routes/auth.$.tsx` — OAuth callback
- `app/lib/billing/products.ts` — three SKUs: \$199 audit, \$499 multi-audit, \$149/\$299 monthly
- `app/middleware/plus-only.ts` — Plus-plan enforcement
- `prisma/schema.prisma` — Shop, Session, Charge tables

Required scopes. `read_script_tags` , `read_orders` , `read_products` , `read_shipping` , `read_discounts` , `read_locations` , `read_checkouts` . No write scopes anywhere — read-only is the architectural commitment.

Acceptance criteria.

- `shopify app dev` runs cleanly and installs on a fresh Plus dev store.
- Non-Plus stores see the gate message and cannot proceed.
- One-time charge of \$199 can be initiated and confirmed via Shopify Managed Billing.
- Recurring \$149/mo subscription can be initiated and shows up in Partners admin.
- Uninstall webhook deletes shop session cleanly.
- Lighthouse meets Built for Shopify performance baseline.

Tests. Plan-detection unit test (Plus vs non-Plus). Billing flow integration test on dev store. Webhook signature verification.

Browser proof. Install on Plus dev store, screenshot embedded dashboard. Install on non-Plus dev store, screenshot the gate. Initiate test charge, screenshot Partners admin.

Rollback. Uninstall from dev store. `git revert` . No production impact.

Codex evaluation questions.

- Does Plus detection survive plan changes (downgrade, trial-to-paid)?
- Are scopes truly minimum-necessary?
- Is the billing flow handling failures (declined card, network error) gracefully?
- Does the app meet Built for Shopify Lighthouse thresholds?

Stop/go. Pass = move to Slice 2.

Time budget. 2–3 days.

Slice 2 — Script Discovery + Classification

Goal. Pull all active Scripts from the merchant’s store and classify them. Even this alone is valuable and demo-able.

Non-goals. No simulation. No diff. No Functions comparison yet. Just inventory.

User-visible behavior. Merchant clicks “Discover my Scripts.” App queries the Shopify Admin API, lists every active Script with: title, type (line item / shipping / payment), Ruby source code (read-only display, syntax-highlighted), last modified date, current published status. Each Script gets an automatic classification badge: `Discount Logic`, `Shipping Rule`, `Payment Customization`, `Market Pricing`, `B2B Logic`, **or** `Other / Needs Review`.

Why this is Slice 2. A merchant who installs the app and sees a clean inventory of their own Scripts in 30 seconds — including ones they’d forgotten about — is already getting value. This is the App Store screenshot. This is the Loom demo. This alone differentiates from the consultant pitch which requires a 30-minute call to even produce this list.

Files likely to change.

- `app/routes/app.scripts._index.tsx` — list view
- `app/routes/app.scripts.$id.tsx` — Script detail with source view
- `app/lib/shopify/scripts.ts` — Admin API wrapper for Script discovery
- `app/lib/classifier/script-classifier.ts` — heuristic classification (regex + AST patterns + LLM fallback)
- `prisma/schema.prisma` — `DiscoveredScript`, `ScriptClassification`

API note. Shopify exposes Scripts via the legacy `Script` admin object (REST + GraphQL where supported). Verify exact shape against current 2026-04 API version

before building. If access patterns have changed, fall back to merchant-paste workflow (textarea where they paste their Script source).

Acceptance criteria.

- All active Scripts on the store appear in the list within 10 seconds of clicking discover.
- Source code displays correctly with syntax highlighting.
- Classification accuracy $\geq 80\%$ on a hand-curated test set of real-world Scripts.
- Manual reclassification override available per-Script.
- Inactive/archived Scripts shown in a separate tab.

Tests. Classifier unit tests against ≥ 20 anonymized real Script samples (gather from public Shopify Community threads + your own dev stores). API integration test on dev store with seeded Scripts.

Browser proof. Screenshot of populated Script list on a dev store seeded with 5 representative Scripts. Screenshot of Script detail view showing classification + source.

Rollback. Feature flag. Postgres migration is additive.

Codex evaluation questions.

- What if a merchant has 50 Scripts? 200?
- Are deactivated/draft Scripts handled differently than active ones?
- Does the source view sanitize against XSS (Ruby code is text but the renderer must not execute anything)?
- What's the fallback if the Script API surface has changed in the current API version?

Stop/go. Pass = move to Slice 3. The Script list view is also demoable to inbound waitlist signups as proof-of-life.

Time budget. 4–6 days.

Slice 3 — Cart Fixture Generation from Order History

Goal. Build a library of representative cart scenarios from the merchant's last 90 days of real orders. These fixtures become the test inputs for everything downstream.

Non-goals. No simulation yet. No diff. Just fixtures.

User-visible behavior. Merchant clicks “Generate fixtures.” App pulls last 90 days of orders, anonymizes customer PII, extracts cart compositions (line items, quantities, customer tags, shipping addresses, market, currency, applied discounts), and produces a deduplicated set of representative fixtures. Each fixture displays as a card: product mix, expected discount applied (from order history), expected shipping rate, expected available payment methods.

Why this matters. This is how you establish the baseline of “what your Scripts actually do.” The merchant doesn’t have to remember or describe Script behavior — the order history tells you what the Script produced for real customers. This sidesteps the need for a Ruby execution sandbox.

Files likely to change.

- `app/routes/app.fixtures._index.tsx`
- `app/lib/fixtures/extractor.ts` — pull orders, anonymize, deduplicate
- `app/lib/fixtures/dedup.ts` — collapse near-identical orders into representative fixtures
- `app/lib/shopify/orders.ts` — paginated order fetching with rate limit handling
- `prisma/schema.prisma` — `CartFixture`, `FixtureBaseline`

Deduplication strategy. Two orders with same products + same customer tags + same shipping country + same applied discount = one fixture. Quantity variations matter; preserve quantity diversity.

PII handling. Strip customer names, emails, phone numbers, addresses (keep country + postal code prefix only). Document this prominently for App Store privacy review.

Acceptance criteria.

- 90 days of orders processed in under 5 minutes for a shop doing 1,000 orders/month.
- Deduplication produces 50–500 representative fixtures (not 10,000 raw orders).
- Each fixture has a captured “baseline” of expected discount, shipping, payment outcomes from the actual order.
- PII fields confirmed absent in stored fixtures (automated scan).
- Rate limit backoff implemented; never trips Shopify’s 2-points/sec throttle.

Tests. Order parser unit tests on real anonymized order JSON. Deduplication algorithm

tests. PII scrubber regex tests with negative cases. Rate limit simulation test.

Browser proof. Screenshot of fixtures list on a dev store with seeded order history. Show one fixture detail with baseline outcomes.

Rollback. Feature flag. Fixture data deletable per-shop.

Codex evaluation questions.

- Is PII genuinely stripped or just hidden?
- What happens with shops that have <10 orders in 90 days?
- How are orders with multiple discount codes (Script + manual) baseline-attributed?
- Is the fixture library re-runnable as new orders arrive?

Stop/go. Pass = move to Slice 4.

Time budget. 5–7 days.

Slice 4 — Migration Risk Audit Report (the \$199 product)

Goal. Generate the deliverable PDF that merchants pay \$199 for. After this slice, the product is monetizable.

Non-goals. No Functions output yet (that's Slice 5+). No live regression. The audit is based on Script-side analysis only at this point.

User-visible behavior. Merchant clicks "Generate Audit." App combines Slice 2 (Script inventory + classification) and Slice 3 (cart fixtures + baselines) into a Migration Risk PDF report containing:

1. **Executive summary** — count of Scripts, count of fixtures, top 3 highest-risk items
2. **Per-Script breakdown** — what it does, classification, complexity score, edge cases observed in last 90 days
3. **Fixture-by-fixture risk table** — each fixture with its baseline outcome and a "things that could go wrong in migration" annotation
4. **Migration checklist** — ordered list of what to test in Functions, prioritized by revenue impact
5. **Open questions for the merchant or their developer** — places where the Script behavior is ambiguous and needs human judgment

Paywall. Generation requires payment via Shopify Managed Billing. \$199 for single-Script-family audit, \$499 for all Script families. After payment, PDF generates and downloads.

Files likely to change.

- `app/routes/app.audit.tsx` — **paywall + generation trigger**
- `app/lib/audit/risk-scorer.ts` — **complexity + risk scoring per Script**
- `app/lib/pdf/audit-report.tsx` — **React-PDF template**
- `app/lib/billing/charge.ts` — **one-time charge wrapper**
- `prisma/schema.prisma` — **AuditReport, AuditPurchase**

Risk scoring heuristics (v1).

- **High:** Scripts with conditional logic referencing customer tags, market, or external state
- **Medium:** Scripts with multi-tier discount logic or shipping rate manipulation
- **Low:** Scripts with simple flat-percentage or fixed-amount logic
- **Unknown:** anything the classifier couldn't categorize confidently

Acceptance criteria.

- PDF generates in under 30 seconds for a store with 5 Scripts and 200 fixtures.
- Charge flow completes via Shopify Managed Billing.
- PDF is professionally laid out (use design tokens; this is the paid deliverable, it has to look good).
- Report is comprehensible to a non-technical Plus ops lead, not just a developer.
- Generated reports are stored and re-downloadable for 12 months.

Tests. PDF snapshot tests for a known fixture set. Charge flow integration test. PDF accessibility check (screen reader compatible).

Browser proof. Generate a real audit on a dev store. Download the PDF. Screenshot the PDF first 3 pages. Show charge in Partners admin.

Rollback. Disable paywall route. Refund any charges via Shopify admin if needed.

Codex evaluation questions.

- Is the PDF actually worth \$199? Show it to 2 Slice 0 waitlist signups before public launch.
- Are the “open questions for the merchant” specific and actionable, or generic?
- Does the executive summary land in 30 seconds of reading?
- Is the report defensible against a refund request?

Stop/go. This is the cash gate. If you can’t sell at least 5 audits in the first 14 days post-launch at \$199, the rest of the roadmap is academic. Pause and re-examine messaging before building Slice 5.

Time budget. 6–8 days.

Slice 5 — Functions Output Capture

Goal. When the merchant has deployed their new Functions (in test or production), capture the Function outputs against the same cart fixtures from Slice 3.

Non-goals. No diff yet. Just capture.

User-visible behavior. Merchant connects their Functions (the app already has the access via Admin API). Merchant clicks “Capture Function outputs.” App runs each fixture through the merchant’s currently-deployed Functions in a sandbox/test mode and records: applied discounts per line item, computed shipping rates, payment methods returned, any Function errors or exceptions.

Technical reality check. Shopify Functions can be invoked in the Functions API for testing. Confirm exact API surface against 2026-04 version. If test invocation is gated, fall back to: (a) capturing real Function outputs from live orders post-deployment, or (b) requiring merchant to run Functions in a dev/staging Plus store where Sentinel observes via webhook.

Files likely to change.

- `app/routes/app.functions._index.tsx` — Function inventory + status
- `app/routes/app.functions.capture.tsx` — capture run UI
- `app/lib/shopify/functions.ts` — Functions API wrapper
- `app/lib/capture/runner.ts` — fixture-by-fixture invocation
- `prisma/schema.prisma` — `FunctionOutput`, `CaptureRun`

Acceptance criteria.

- Capture run completes for 200 fixtures in under 10 minutes.
- Function exceptions are captured and surfaced (not swallowed).
- Output schema matches the Script baseline schema from Slice 3 (so they can be diffed in Slice 6).
- Re-running captures preserves history for trend analysis.

Tests. Functions API integration test against a real test Function. Schema-match unit tests between capture output and baseline format.

Browser proof. Capture run on a dev store with deployed test Functions. Screenshot the captured output table.

Rollback. Feature flag. Capture data deletable per-shop.

Codex evaluation questions.

- What if the merchant hasn't deployed any Functions yet?
- How are Function errors distinguished from "Function says no discount applies"?
- Is the capture API rate-limit-safe for shops with 1,000+ fixtures?

Stop/go. Pass = move to Slice 6.

Time budget. 5–7 days.

Slice 6 — Diff Engine + Drift Alerts

Goal. Compare Script baseline (from Slice 3) against Function output (from Slice 5) per fixture. Surface mismatches.

Non-goals. No automatic remediation. The product reports drift; the merchant fixes it.

User-visible behavior. Dashboard shows a table of fixtures with a status column:

`Match`, `Drift Detected`, `Function Error`. Clicking a row opens a side-by-side diff:

Script baseline (left) vs Function output (right) with mismatched fields highlighted.

Filters: show only drifts, show only errors, group by Script type.

Diff types per customization.

- **Discount:** line-item-level diff of discount amount, target, allocation

- **Shipping:** rate-code-level diff of available rates, prices, conditions
- **Payment:** method-availability diff with reasons
- **Market pricing:** per-variant per-market price diff

Drift severity scoring.

- **Critical:** discount amount differs by >\$1 or >5%; shipping rate differs by >\$1; payment method appearance/disappearance
- **Warning:** sort order differs, label differs, no functional impact
- **Info:** Function added a benefit Script didn't (often safe but worth review)

Files likely to change.

- `app/routes/app.diff._index.tsx` — diff dashboard
- `app/routes/app.diff.$fixtureId.tsx` — per-fixture detail
- `app/lib/diff/discount-diff.ts`
- `app/lib/diff/shipping-diff.ts`
- `app/lib/diff/payment-diff.ts`
- `app/lib/diff/severity.ts`
- `prisma/schema.prisma` — DiffResult

Acceptance criteria.

- Diff completes for 200 fixtures in under 30 seconds.
- All four diff types implemented with unit-tested correctness.
- Severity scoring matches a curated golden dataset.
- Side-by-side UI clearly highlights only the mismatched fields.

Tests. Diff algorithm unit tests with hand-curated input/output pairs covering ≥ 30 mismatch patterns. UI snapshot tests.

Browser proof. Generate diffs on a dev store with intentionally-different Scripts and Functions. Screenshot dashboard + per-fixture detail.

Rollback. Feature flag.

Codex evaluation questions.

- Are floating-point differences handled (rounding tolerance)?
- Is the severity scorer overly noisy or too forgiving?
- Does the UI scale to 1,000+ fixtures without becoming unusable?

Stop/go. Pass = move to Slice 7.

Time budget. 6–8 days.

Slice 7 — Continuous Regression Suite (the \$149/mo product)

Goal. After migration cutover, keep running the diff on a schedule. Alert merchant on new drift.

Non-goals. No Function authoring. No remediation suggestions beyond “this fixture drifted on this date.”

User-visible behavior. Merchant subscribes to the Regression Suite tier (\$149/mo). Cron runs nightly: re-execute Function capture for all fixtures, re-diff against baseline, store result. New drift triggers an email + in-app alert: “3 fixtures drifted today — promotion code FALL10 now applies \$2 less than before.” Dashboard shows drift trend over time.

Subscription gating. Diff history older than 7 days is gated behind the Regression Suite subscription. Free post-audit users see only the most recent diff snapshot.

Files likely to change.

- `app/jobs/nightly-regression.ts` — cron handler
- `app/routes/app.regression._index.tsx` — trend dashboard
- `app/lib/email/drift-alert.tsx` — React Email template
- `app/lib/billing/subscription.ts` — monthly recurring charge
- `prisma/schema.prisma` — `RegressionRun`, `DriftAlert`

Fixture refresh policy. Fixtures regenerate weekly from the latest 90 days of orders, capturing new edge cases as they appear in real traffic.

Acceptance criteria.

- Nightly cron runs reliably for a week without manual intervention.

- Drift alerts fire only on critical/warning severity, not info-level noise.
- Subscription charge succeeds via Shopify Managed Billing.
- Alert email arrives within 30 minutes of drift detection.
- Trend chart shows drift count over the last 30 days.

Tests. Cron schedule test. Email render snapshot. Subscription billing integration.

Browser proof. Trigger a manual regression run, intentionally introduce drift, screenshot the alert email + dashboard.

Rollback. Cron pause. Subscription cancelable from Shopify admin.

Codex evaluation questions.

- What happens when a merchant edits their Function and drift increases legitimately?
- Is alert fatigue managed (don't email every night about the same drift)?
- Does the trend chart update in real time or on cron?

Stop/go. Pass = move to Slice 8.

Time budget. 5–7 days.

Slice 8 — Multi-Customization Expansion (the \$299/mo product)

Goal. Extend coverage from discount Scripts to shipping rules, payment customizations, B2B price lists, and market-specific logic. Justify the higher tier.

Non-goals. No new product surface. Same diff engine, more customization types.

User-visible behavior. Subscribers to the \$299/mo tier see additional customization types in the Script inventory, fixture generation, and diff dashboards. The same paid-upgrade flow reveals the additional coverage.

Customization types added.

- Carrier service customizations (custom shipping rate logic)
- Payment customizations (method visibility, ordering)
- B2B price lists and customer-specific catalog overrides

- Markets-specific pricing rules
- Checkout UI extensions that modify cart state (where observable via Functions)

Files likely to change.

- `app/lib/classifier/*` — extend to new customization types
- `app/lib/fixtures/*` — include shipping/payment outcomes in baseline
- `app/lib/diff/*` — extend diff engine
- `prisma/schema.prisma` — CustomizationType enum expanded

Acceptance criteria.

- All five additional customization types covered with their own classifier, fixture extractor, capture, and diff.
- \$299/mo tier paywalls correctly.
- Existing \$149/mo subscribers can upgrade with prorated billing.

Tests. Per-customization-type unit tests. Cross-type integration test.

Browser proof. Plus dev store with all five customization types active, full audit + regression dashboard showing each.

Rollback. Per-type feature flags.

Codex evaluation questions.

- Does the audit PDF accommodate the expanded scope without becoming unreadable?
- Are upgrade flows tested edge-to-edge?
- Is there a clear differentiator the merchant feels at the \$299 tier?

Stop/go. Pass = move to Slice 9.

Time budget. 6–8 days.

Slice 9 — Launch: App Store Submission + Plus Partner Distribution

Goal. Submit to Shopify App Store, get listed in the Plus apps directory, pass review on

first attempt.

Non-goals. No new features.

User-visible behavior. First-install onboarding: 2-step welcome (verify Plus, run discover), clear path to first audit. Empty states. Loading states. Error boundaries. GDPR mandatory webhooks. App listing with Plus-targeted screenshots and 60-second demo video.

Files likely to change.

- `app/routes/app.welcome.tsx`
- `app/routes/webhooks.gdpr.*.tsx`
- `public/legal/privacy.html`
- App Store listing assets in `/marketing`

Acceptance criteria.

- App Store review checklist: all green.
- Lighthouse meets Built for Shopify thresholds.
- All 3 GDPR webhooks respond correctly.
- Demo video shows: install → discover Scripts → generate fixtures → download audit PDF in under 90 seconds.
- Plus-specific positioning in listing copy (price tier, urgency, deadline).

Tests. GDPR webhook signature tests. Onboarding E2E test. Lighthouse CI in pipeline.

Browser proof. Full install + audit + PDF download recorded as demo video. Lighthouse report. Listing preview.

Rollback. Unpublish from App Store if critical bug.

Codex evaluation questions.

- Is the listing copy specifically Plus-targeted or generic?
- Does the demo video communicate the wedge (parity testing, not Function authoring)?
- Are all required Shopify App Store fields complete?
- Have you submitted to Built for Shopify program?

Stop/go. Pass = submit. Fail = fix and resubmit.

Time budget. 5–7 days, including App Store review wait time.

Total time budget

Slice	Days
0 — Distribution Validation	7
1 — App Shell + Billing	2–3
2 — Script Discovery	4–6
3 — Cart Fixtures	5–7
4 — Audit Report (cash gate)	6–8
5 — Functions Capture	5–7
6 — Diff Engine	6–8
7 — Regression Suite	5–7
8 — Multi-Customization	6–8
9 — Launch	5–7

Total: 51–68 days of build time (Slice 0 runs concurrent with prep, not blocking).

Critical path: Slices 1–4 must complete by **May 13, 2026** (14 days from today) to capture the panic window between April 15 (editing block) and June 30 (cutoff). Slices 5–7 can ship through May and June. Slice 8 is post-cutoff retention work.

Parallel work paths:

- Slice 0 distribution validation runs in days 1–7 alongside Slice 1 build
- App Store submission can begin as early as end of Slice 4 (audit-only MVP is shippable)
- Slices 5–8 ship as updates to the live App Store listing

What can kill the timeline

1. **Shopify Scripts API surface has changed.** Verify in Slice 2; have merchant-paste fallback ready.
2. **Shopify Functions test invocation is gated.** Plan for live-order observation as fallback in Slice 5.
3. **Audit PDF doesn't sell at \$199.** Slice 4 stop/go gate catches this. Don't build Slice 5 if Slice 4 doesn't convert.
4. **Distribution costs exceed unit economics.** Slice 0 catches this. CPA above \$50 for an audit-only buyer is unsustainable.
5. **Shopify ships native parity testing in May or June.** Low probability, high impact. Mitigate by accelerating App Store launch.

What is explicitly not in the roadmap

- Function authoring or migration code generation (Function Junction territory)
- Discount design UI (Discount Ninja, Regios)
- Auto-conversion of Ruby to Functions (technically infeasible at quality bar)
- Non-Plus support
- Other platforms (BigCommerce, Adobe Commerce, etc.)
- Generic checkout monitoring (Elevar, Analyzify)
- Promotion strategy advice (consultant territory; we are the QA tool, not the strategist)

If a feature request comes in for any of these: "Not in scope. We are the regression test layer. Here are three apps that do that other thing." Saying no is the wedge.